Tutorial 13: Denoising AutoEncoders

Laiba Niazi

536672

1. Objective

The primary goal of this experiment is to implement and evaluate three distinct autoencoder architectures using the **MNIST dataset**: a **Denoising Autoencoder (DAE)**, a **Deep Autoencoder**, and a **Variational Autoencoder (VAE)**. The study focuses on their ability to reconstruct clean images from noisy inputs and learn meaningful latent representations.

2. Dataset and Preprocessing

- **Dataset**: The standard MNIST dataset of handwritten digits.
- **Normalization**: Images are converted into PyTorch tensors with a resolution of 28x28 pixels.
- **Noise Injection**: For the denoising task, random Gaussian noise is added to the images using a **noise factor of 0.5**, followed by value clipping between [0, 1].
- **Hardware**: The implementation utilizes **CUDA** for GPU acceleration to optimize training time.

3. Methodology & Architectures

3.1 Denoising Autoencoder (DAE)

The DAE utilizes a convolutional approach specifically designed to learn spatial filters for noise removal.

- **Encoder**: Consists of two convolutional layers (32and 64 filters) using 3x3 kernels, followed by **MaxPool2d** layers to reduce spatial dimensions.
- **Decoder**: Reconstructs the image using convolutional layers and **Upsampling** (scale factor of 2). A **Sigmoid** activation is used at the output to ensure pixel values remain in the [0, 1] range.

```python
class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super(DenoisingAutoencoder, self).__init__()
        # Encoder[cite: 1]
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), # Conv2D 32
            nn.MaxPool2d(2, 2),                         # Max Pooling
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),# Conv2D 64
            nn.MaxPool2d(2, 2)                          # Max Pooling
        )
        # Decoder[cite: 1]
        self.decoder = nn.Sequential(
            nn.Conv2d(64, 64, 3, padding=1), nn.ReLU(),
            nn.Upsample(scale_factor=2),                # UpSampling
            nn.Conv2d(64, 32, 3, padding=1), nn.ReLU(),
            nn.Upsample(scale_factor=2),                # UpSampling
            nn.Conv2d(32, 1, 3, padding=1), nn.Sigmoid()# Final reconstruction
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

dae_model = DenoisingAutoencoder().to(device)
```

3.2 Deep Autoencoder (Task 1)

The Deep Autoencoder is a fully connected "bottleneck" network.

- **Encoder**: Flattens the 28x28 image into a 784-unit vector, passing through linear layers of sizes 512, 256, and 128 to reach a compressed latent space of **64**.
- **Decoder**: Mirrors the encoder to expand the vector back to 784 units before reshaping it to its original 28x28 dimensions.

```python
class DeepAutoencoder(nn.Module):
    def __init__(self):
        super(DeepAutoencoder, self).__init__()
        # Deep stack of linear layers
        self.encoder = nn.Sequential(
            nn.Linear(28*28, 512), nn.ReLU(),
            nn.Linear(512, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 64) # Compressed latent space
        )
        self.decoder = nn.Sequential(
            nn.Linear(64, 128), nn.ReLU(),
            nn.Linear(128, 256), nn.ReLU(),
            nn.Linear(256, 512), nn.ReLU(),
            nn.Linear(512, 28*28), nn.Sigmoid()
        )

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = self.encoder(x)
        x = self.decoder(x)
        return x.view(-1, 1, 28, 28)

deep_ae_model = DeepAutoencoder().to(device)
```

3.3 Variational Autoencoder (VAE) (Task 2)

The VAE introduces a probabilistic latent space rather than a fixed vector.

- **Encoder**: Maps the input to two vectors—**mu** (mean) and **logvar** (log-variance)—representing a distribution.

- **Reparameterization**: Uses the reparameterization trick (mu + eps x std) to allow backpropagation through stochastic sampling.
- **Decoder**: Samples from the latent space (dimension = 20) to reconstruct the original digit.

```python
class VAE(nn.Module):
    def __init__(self, latent_dim=20):
        super(VAE, self).__init__()
        self.fc1 = nn.Linear(784, 400)
        self.fc_mu = nn.Linear(400, latent_dim)
        self.fc_logvar = nn.Linear(400, latent_dim)
        self.fc3 = nn.Linear(latent_dim, 400)
        self.fc4 = nn.Linear(400, 784)

    def encode(self, x):
        h1 = F.relu(self.fc1(x))
        return self.fc_mu(h1), self.fc_logvar(h1)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h3 = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h3))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 784))
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

vae_model = VAE().to(device)
```

4. Training Configuration

All models were trained simultaneously to allow for a direct comparison of convergence:

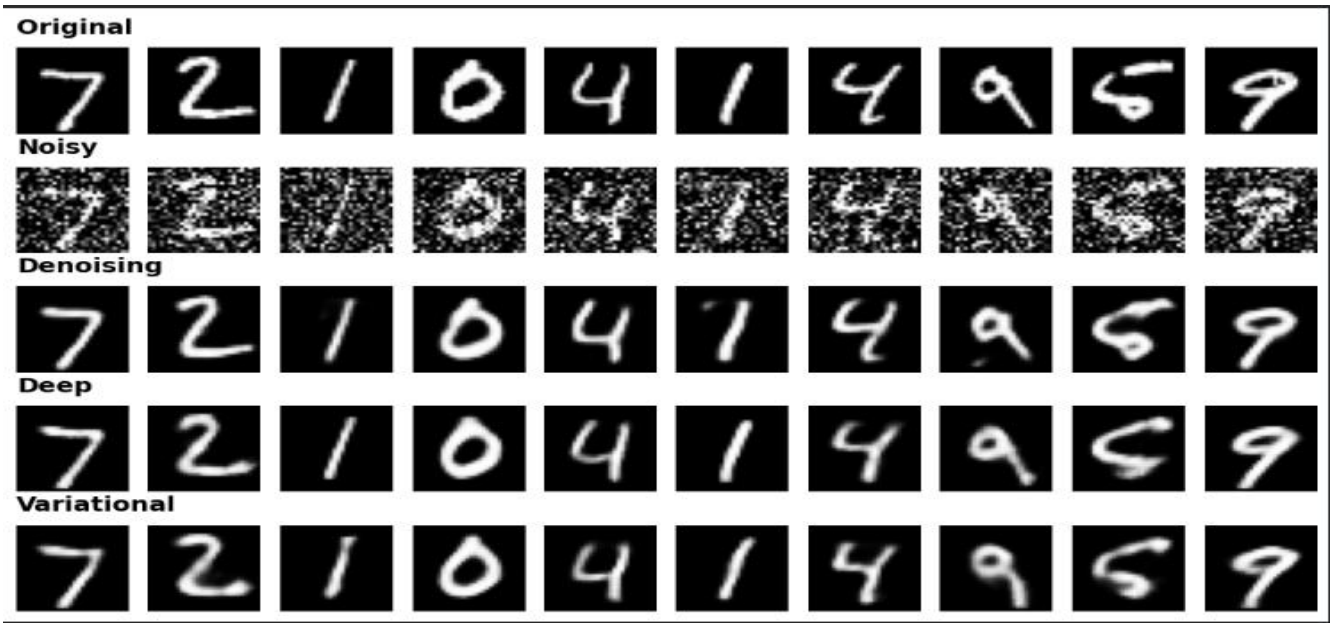
- **Optimizer**: Adam Optimizer for all models.
- **Loss Functions**:
    - **DAE/Deep**: Binary Cross Entropy (BCE).
    - **VAE**: Combined BCE and **Kullback-Leibler Divergence (KLD)** to regularize the latent space.
- **Epochs**: 10.

```
Epoch 1: DAE Loss: 0.1115 | Deep Loss: 0.1445 | VAE Loss: 166.1870
Epoch 2: DAE Loss: 0.1064 | Deep Loss: 0.1281 | VAE Loss: 121.6704
Epoch 3: DAE Loss: 0.1035 | Deep Loss: 0.1190 | VAE Loss: 114.9269
Epoch 4: DAE Loss: 0.1015 | Deep Loss: 0.1115 | VAE Loss: 111.8669
Epoch 5: DAE Loss: 0.1000 | Deep Loss: 0.1063 | VAE Loss: 110.0903
Epoch 6: DAE Loss: 0.0990 | Deep Loss: 0.1022 | VAE Loss: 108.8889
Epoch 7: DAE Loss: 0.0983 | Deep Loss: 0.0993 | VAE Loss: 108.0267
Epoch 8: DAE Loss: 0.0976 | Deep Loss: 0.0969 | VAE Loss: 107.3656
Epoch 9: DAE Loss: 0.0971 | Deep Loss: 0.0948 | VAE Loss: 106.8702
Epoch 10: DAE Loss: 0.0966 | Deep Loss: 0.0931 | VAE Loss: 106.3762
```

5. Performance Analysis

5.1 Visual Reconstruction Results

The qualitative performance is evaluated by passing noisy test images through the models and comparing them to the ground truth.



5.2 Observations

- **DAE Efficiency**: The DAE successfully eliminates heavy Gaussian noise, restoring the structural integrity of the digits.
- **Deep AE Comparison**: While the Deep Autoencoder provides strong reconstruction on clean data, it lacks the spatial awareness of the convolutional DAE when handling noisy inputs.
- **VAE Characteristics**: The VAE reconstructions are generally smoother, reflecting the learned probabilistic distributions in the latent space.

6. Conclusion

The experiment confirms that **Convolutional Denoising Autoencoders** are superior for restoring signals corrupted by noise. Furthermore, **Variational Autoencoders** provide a robust mathematical framework for learning latent distributions, which is essential for generative tasks compared to the deterministic reconstruction of standard Deep Autoencoders.